

Housing Price Prediction Model Comparison Report

Housing Price Prediction — Linear Regression vs. Decision Tree vs. Random Forest

A comprehensive regression case study for a Delhi-region real estate company, comparing three modeling approaches — Multiple Linear Regression (with rigorous manual feature selection), a Decision Tree Regressor, and a Random Forest Regressor — to predict house prices from property characteristics.

1. Project Objective

A real estate company wants to optimize property sale prices using its historical data. This project:

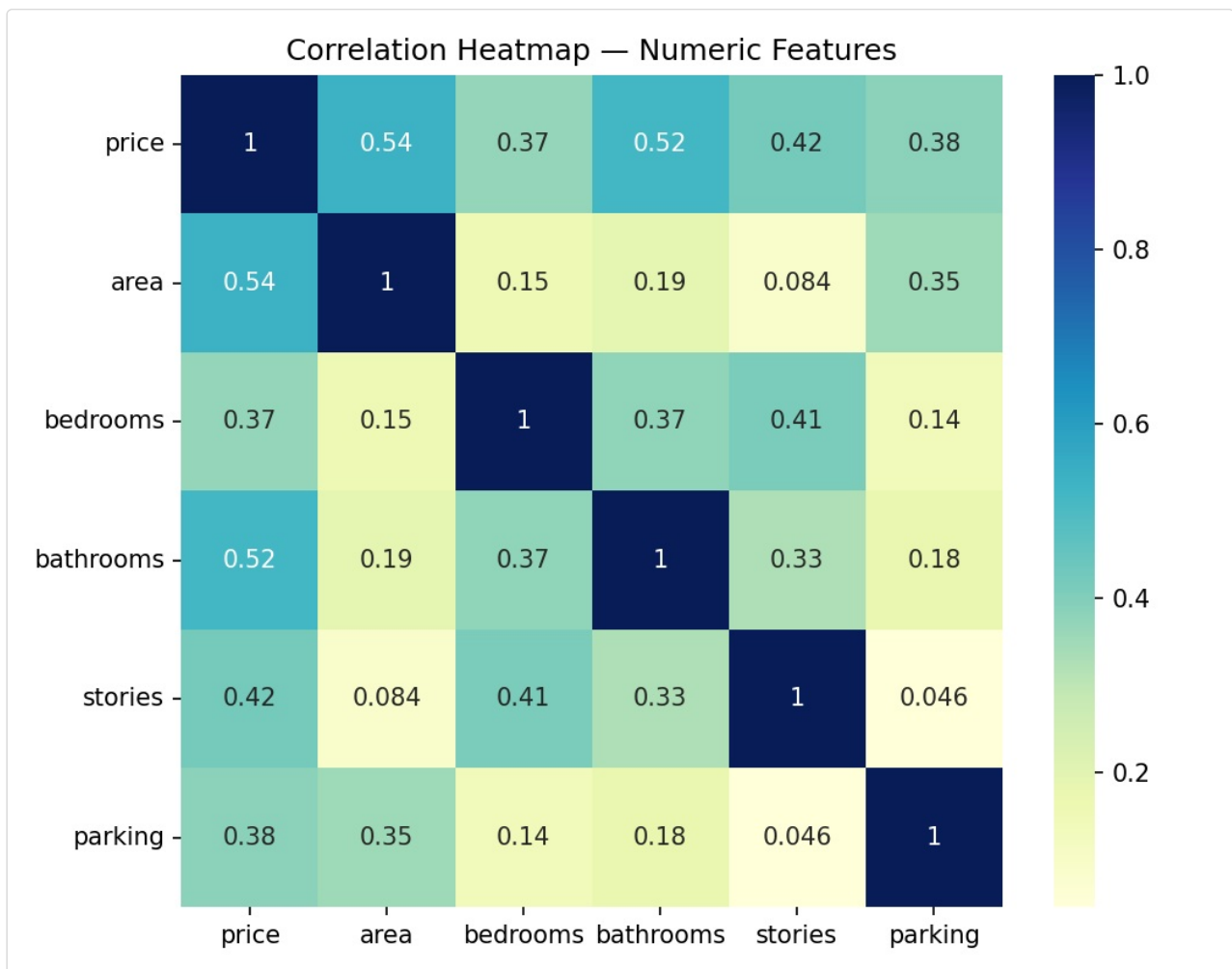
- 1. Identifies which variables (area, bedrooms, bathrooms, amenities, etc.) most affect house price.
- 2. Builds a model that **quantitatively relates price to these variables**, so the business can set data-driven prices for new listings.
- 3. Compares a classical statistical approach (linear regression with formal significance testing) against two modern tree-based methods to see which generalizes best.

2. Dataset

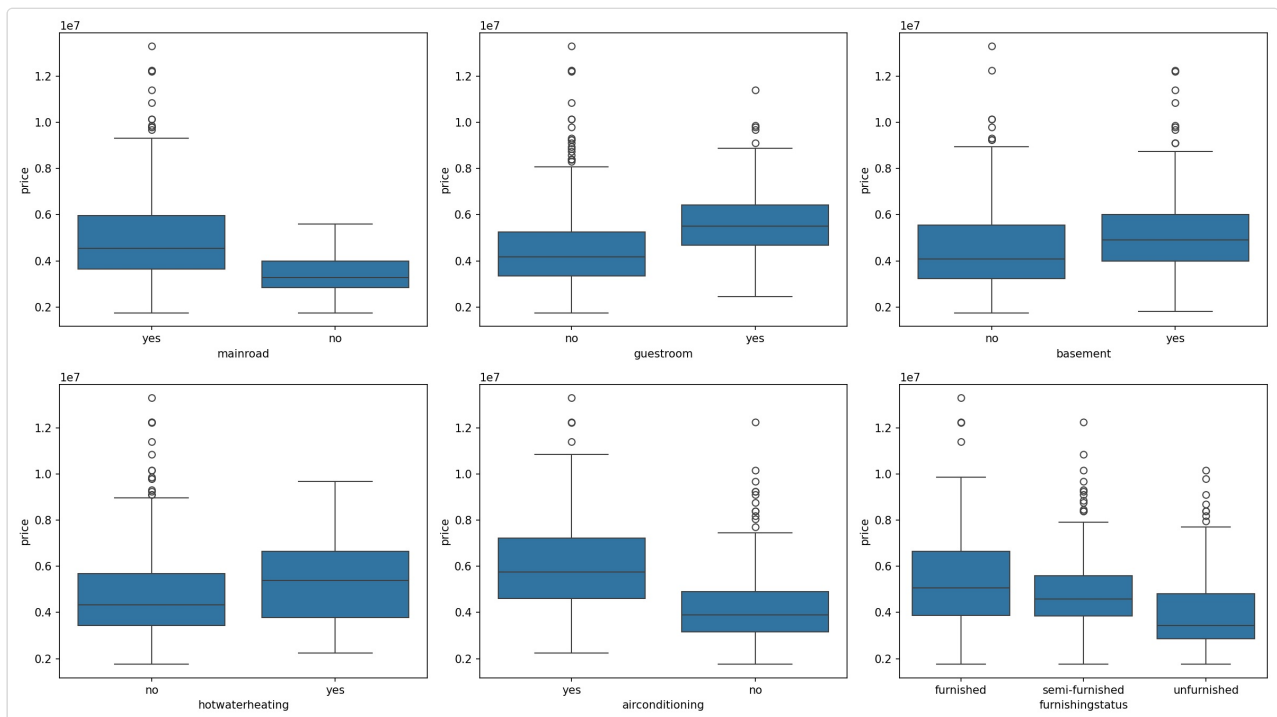
Attribute	Detail
File	Housing.xlsx
Observations	545 properties
Missing values	None
Target	price

Features: area, bedrooms, bathrooms, stories, parking (numeric); mainroad, guestroom, basement, hotwaterheating, airconditioning, prefarea (binary yes/no); furnishingstatus (3-level categorical: furnished / semi-furnished / unfurnished).

3. Exploratory Data Analysis



`area` shows the strongest correlation with `price` among the numeric features, making it the natural starting point for building up the regression model.



Several categorical amenities show a visible price premium — properties on a main road, with air conditioning, or in a preferred area consistently command higher prices, motivating their inclusion as model features rather than treating price as driven by size alone.

4. Tools & Libraries

- **Python 3** — `pandas`, `numpy`
- Ordinary Least Squares regression with manual p-value and t-statistic computation (standard OLS formulas via `numpy` / `scipy`, used here in place of `statsmodels` for portability)
- `scikit-learn` — `train_test_split`, `MinMaxScaler`, `DecisionTreeRegressor`, `RandomForestRegressor`, `r2_score`, and a manual Variance Inflation Factor (VIF) calculation
- `matplotlib`, `seaborn` — visualization

5. Part A — Multiple Linear Regression

5.1 Data Preparation

1. **Binary-encoded** the six yes/no columns (`mainroad`, `guestroom`, `basement`, `hotwaterheating`, `airconditioning`, `prefarea`) as 1/0.
2. **One-hot encoded** `furnishingstatus` (3 levels) into two dummy columns (`semi-furnished`, `unfurnished`), dropping `furnished` as the reference level.
3. **70/30 train-test split** (`random_state=100`).
4. **Min-Max scaled** the numeric columns (`area`, `bedrooms`, `bathrooms`, `stories`, `parking`, `price`) to a 0–1 range, fit on the training set only.

5.2 Iterative Model Building

The notebook builds the model up incrementally, adding one variable at a time to show how explanatory power grows:

Model	Variables	R ²	Adjusted R ²
1	<code>area</code> only	0.283	0.281
2	+ <code>bathrooms</code>	0.477	0.475
3	+ <code>bedrooms</code>	~0.49	~0.49
4 (all 13 features)	All available features	0.681	0.670

5.3 Refining with p-values and VIF

Starting from all 13 features, variables were removed one at a time based on **statistical insignificance (high p-value)** and **multicollinearity (high VIF)**:

Step	Variable Dropped	Reason
1	<code>semi-furnished</code>	p-value = 0.938 (highly insignificant)
2	<code>bedrooms</code>	p-value = 0.206 (insignificant once other features present)
3	<code>basement</code>	p-value = 0.030 (borderline, removed for a cleaner, more robust final model)

Final model — 10 features, all VIF < 1.4 (well within the acceptable range) and all p-values < 0.01:

Feature	Coefficient	p-value
<code>area</code>	+0.2335	<0.0001
<code>bathrooms</code>	+0.2019	<0.0001
<code>stories</code>	+0.1081	<0.0001
<code>mainroad</code>	+0.0497	<0.001
<code>guestroom</code>	+0.0402	0.002
<code>hotwaterheating</code>	+0.0876	<0.0001
<code>airconditioning</code>	+0.0682	<0.0001

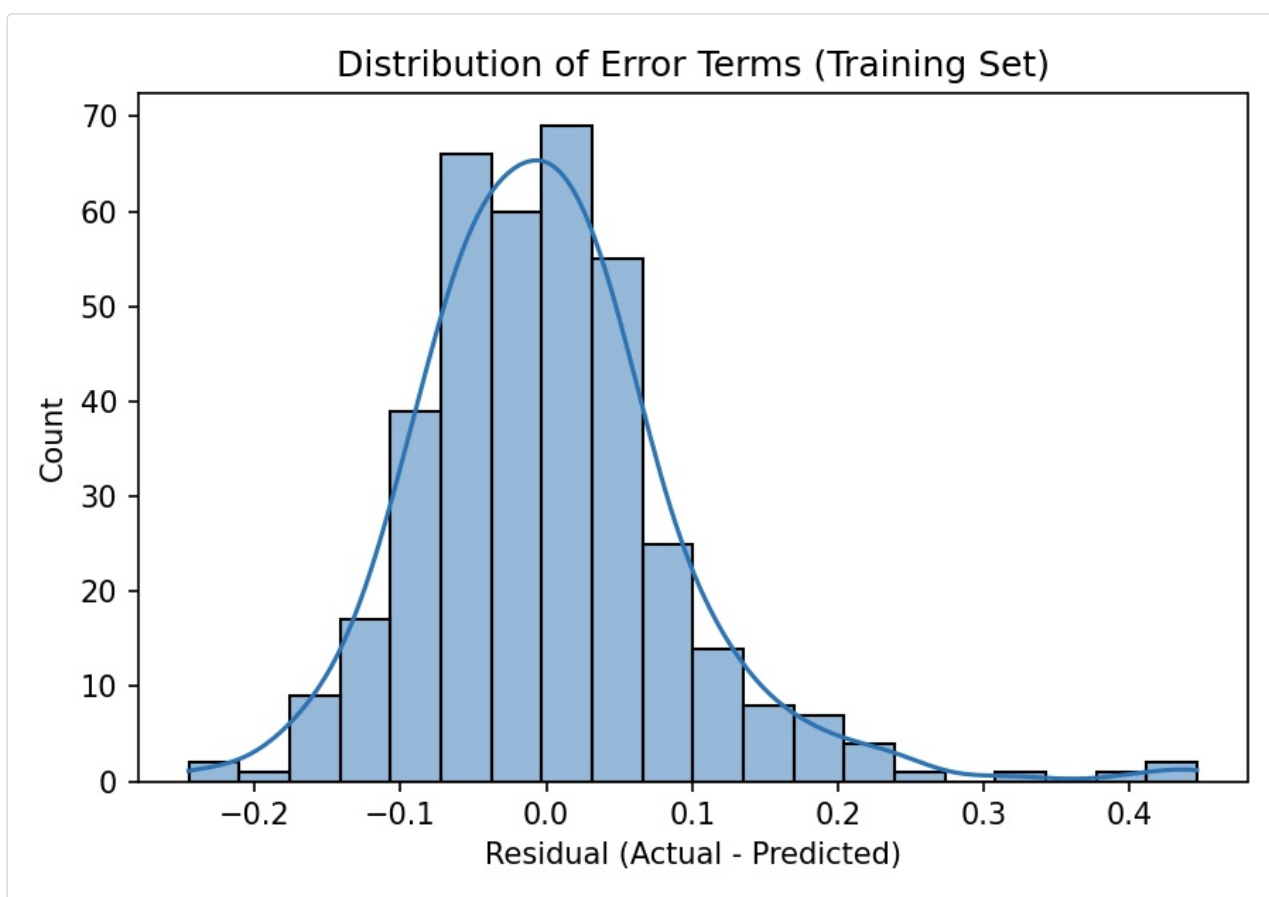
parking	+0.0629	<0.001
prefarea	+0.0637	<0.0001
unfurnished	-0.0337	0.001

Final fitted equation (on Min-Max scaled variables):

```
price = 0.043 + 0.2335·area + 0.2019·bathrooms + 0.1081·stories + 0.0497·mainroad
        + 0.0402·guestroom + 0.0876·hotwaterheating + 0.0682·airconditioning
        + 0.0629·parking + 0.0637·prefarea - 0.0337·unfurnished
```

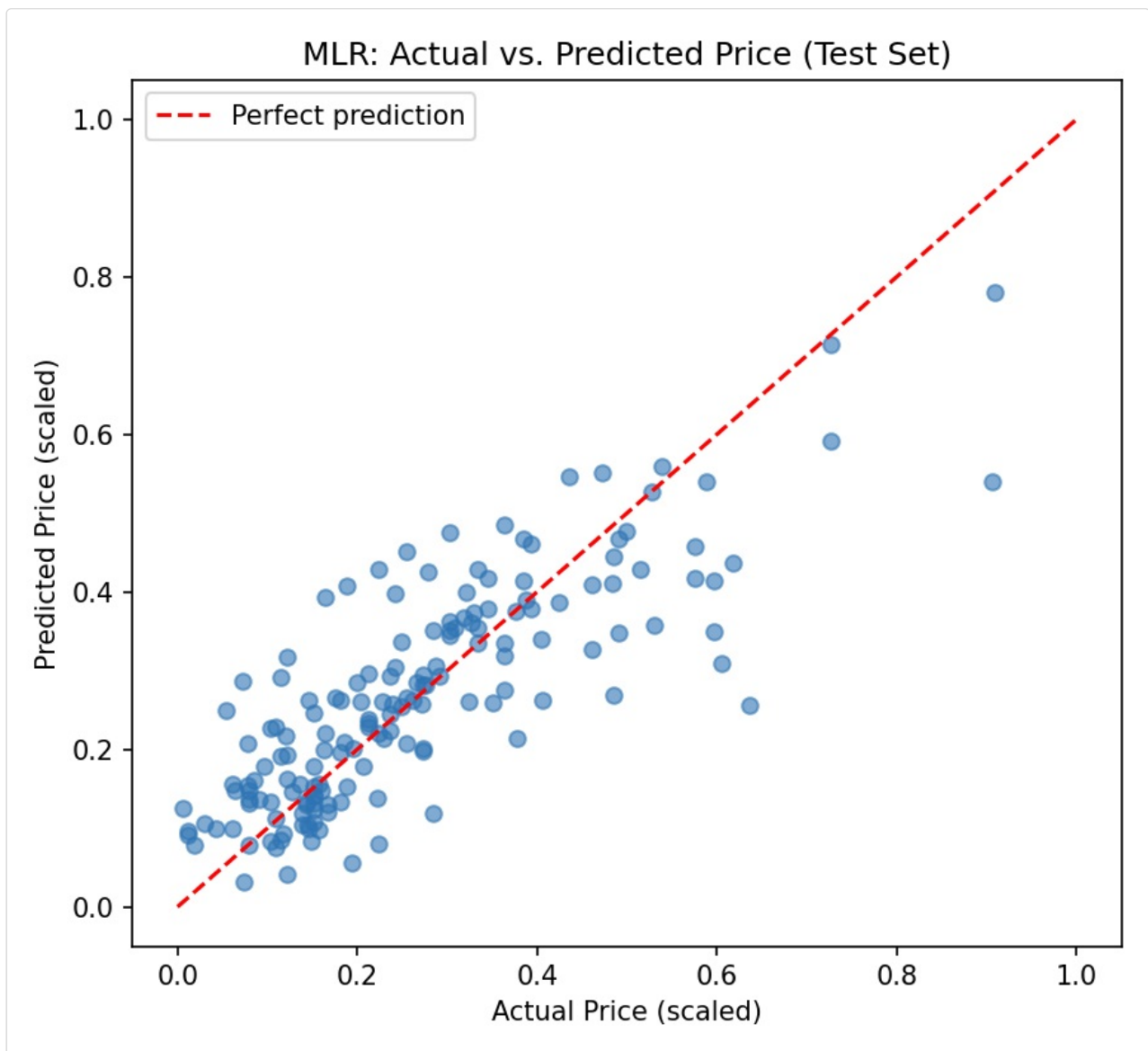
Model fit: Train $R^2 = 0.676$, Adjusted $R^2 = 0.667$.

5.4 Residual Analysis



The residuals are approximately normally distributed and centered near zero, satisfying one of the key assumptions of linear regression and supporting the validity of the p-value-based feature selection performed above.

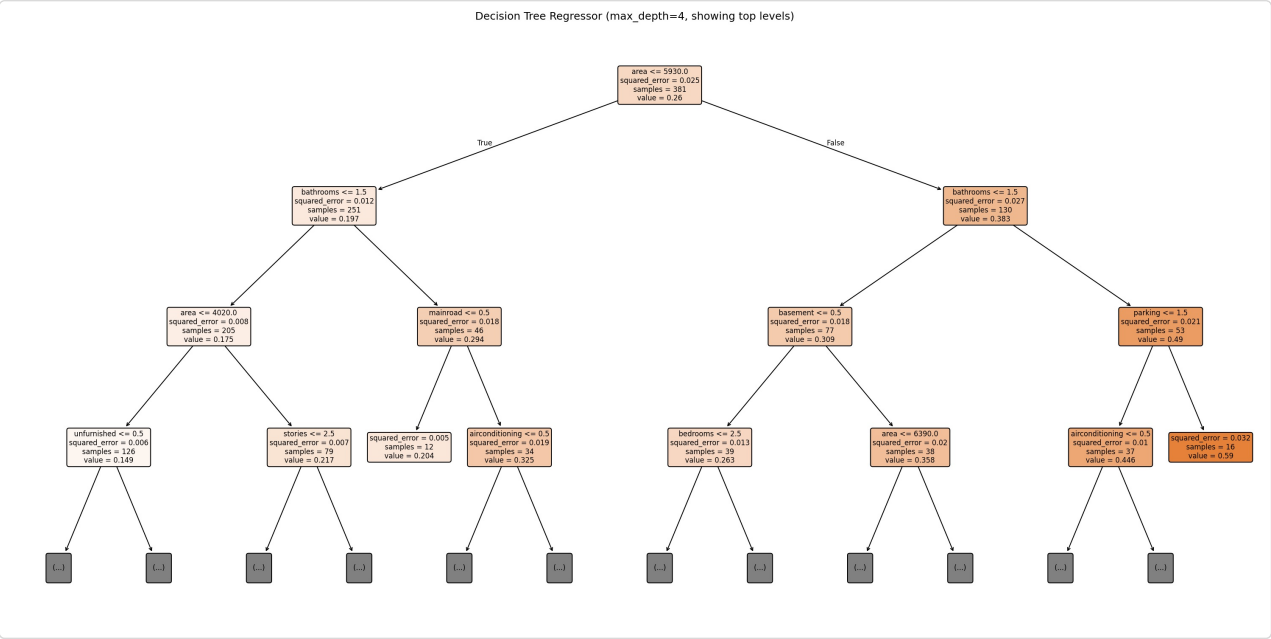
5.5 Test Set Performance



Test $R^2 = 0.660$ — very close to the training R^2 of 0.676, indicating the model **generalizes well** and is not overfitting despite the fairly aggressive feature elimination process.

6. Part B — Decision Tree Regressor

A `DecisionTreeRegressor` (`max_depth=4` , `min_samples_leaf=10`) was fit on the same (unscaled, except target) feature set.



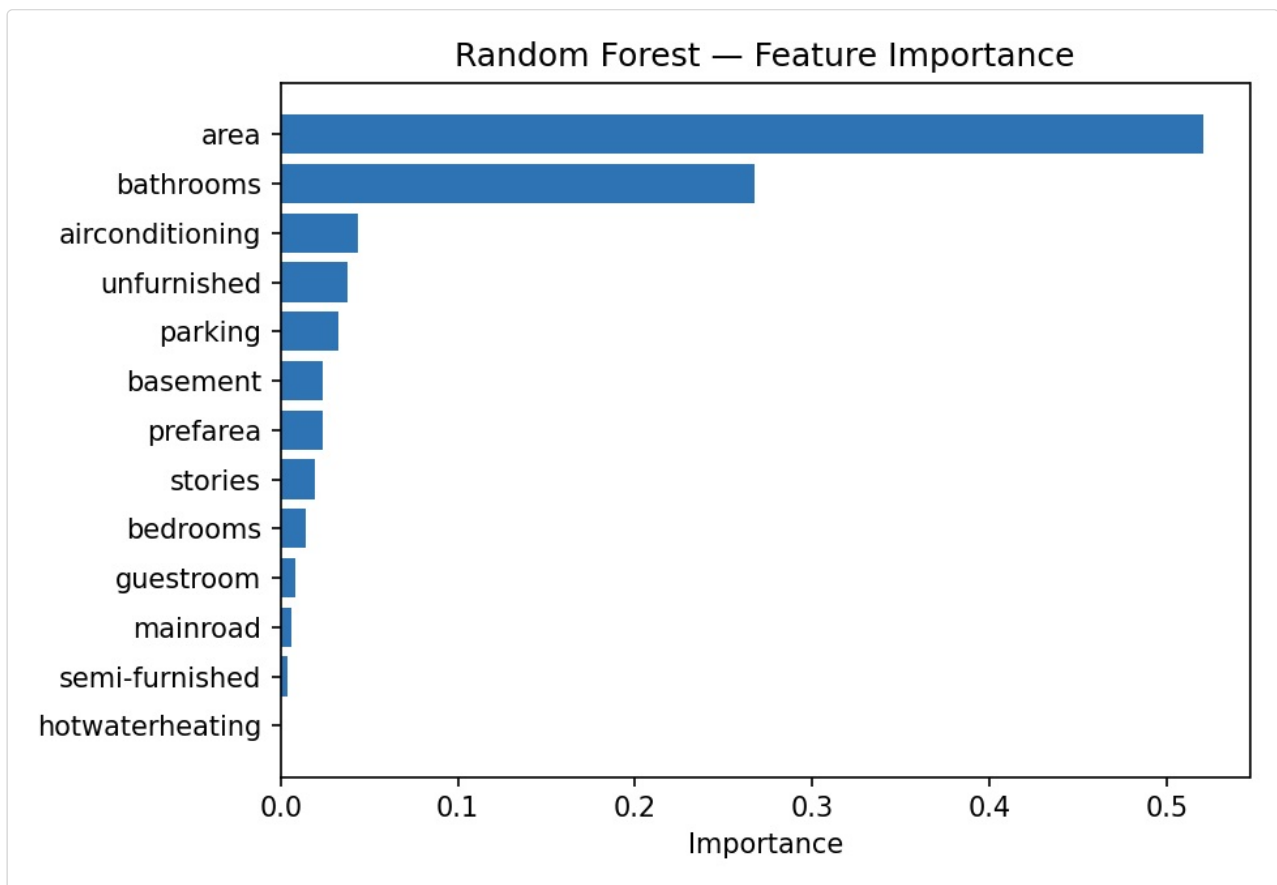
Metric	Train R ²	Test R ²
Decision Tree	0.623	0.529

Feature importance: `area` (56.6%) and `bathrooms` (26.3%) dominate the tree's splits, together accounting for over 80% of its decision-making — several features (`guestroom` , `hotwaterheating` , `prefarea` , `semi-furnished`) receive **zero importance**, meaning the tree never found a useful split on them at this depth.

7. Part C — Random Forest Regressor

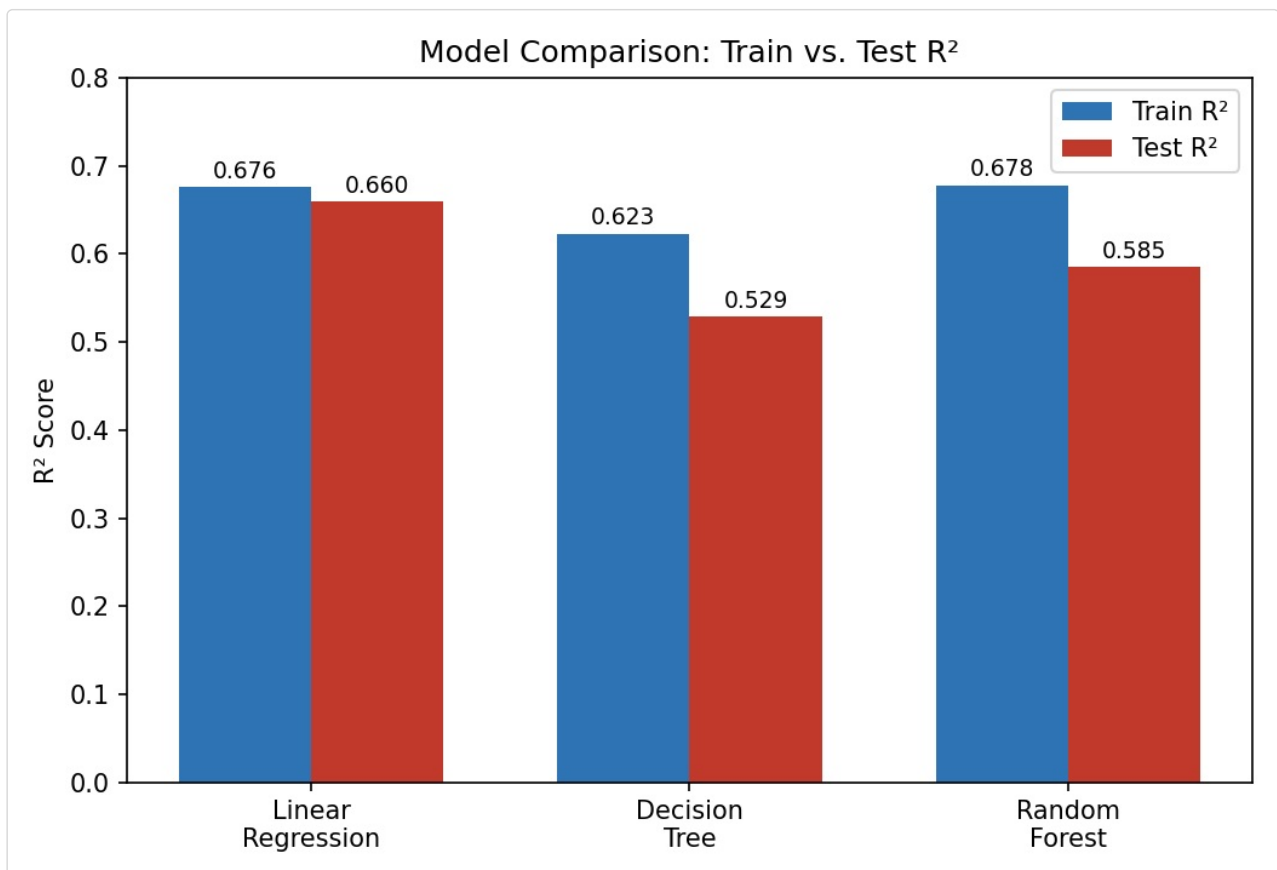
A `RandomForestRegressor` (100 trees, `max_depth=5` , `min_samples_leaf=10`) was fit on the same features.

Metric	Train R ²	Test R ²
Random Forest	0.678	0.585



The Random Forest's feature importances tell a similar story to the single tree — `area` (52.1%) and `bathrooms` (26.7%) dominate — but the ensemble spreads a bit more importance across secondary features (`airconditioning` , `unfurnished` , `parking`) compared to the single tree, and no feature receives exactly zero importance.

8. Model Comparison — The Key Finding



Model	Train R ²	Test R ²	Train-Test Gap
Multiple Linear Regression	0.676	0.660	0.016
Decision Tree	0.623	0.529	0.094
Random Forest	0.678	0.585	0.093

The carefully-refined Multiple Linear Regression model outperforms both tree-based models on the held-out test set — a result worth highlighting rather than assuming "more advanced" models automatically win. Three factors likely explain this:

1. **The dataset is moderately sized (545 rows)** — tree-based models, especially ensembles, typically need more data to realize their advantage over a well-specified linear model.
2. **The relationships in this data appear largely linear** — most numeric features (`area` , `bathrooms` , `stories`) show a fairly proportional relationship with price, which is exactly the setting where linear regression excels.
3. **The linear model benefited from careful, statistically-grounded feature selection** (p-values + VIF), effectively hand-tuning it to this dataset's structure, while the tree-based models used only basic depth/leaf-size constraints without similar tuning.

Both tree-based models also show a **larger train-test gap** (~0.09) than the linear model (0.016), suggesting mild overfitting relative to linear regression's tighter generalization here.

9. Key Insights

- `Area` and `bathrooms` are the two dominant price drivers across all three modeling approaches — this consistency across very different algorithms is a strong signal that the finding is real, not a modeling artifact.
- **A well-specified linear model can beat tree ensembles on small-to-medium, largely-linear tabular datasets** — this project is a concrete counterexample to the common assumption that ensemble methods are strictly superior.
- **Statistical rigor (p-values, VIF) added real value** — the disciplined, one-variable-at-a-time elimination process produced a final model with excellent generalization (only a 0.016 train-test R² gap), demonstrating the payoff of careful feature selection over simply throwing all features at a model.
- **Amenities carry real, statistically significant price premiums** — air conditioning, hot water heating, a preferred location, and main-road access all remained significant even after controlling for size and room counts, giving the real estate company concrete, actionable pricing levers beyond just square footage.

10. Limitations & Next Steps

- **No hyperparameter tuning** was performed for the Decision Tree or Random Forest (fixed `max_depth / min_samples_leaf`); a `GridSearchCV` sweep could narrow or even close the gap with linear regression.
- **No cross-validation** — a single 70/30 split was used throughout; k-fold CV would give more robust R² estimates for all three models.
- **No interaction terms or polynomial features** were tested in the linear model, which could capture non-linear effects (e.g., diminishing returns on very large `area`) that the tree-based models might otherwise be expected to exploit better.
- **A natural next step:** try Gradient Boosting (e.g., XGBoost/LightGBM), which often narrows the gap with linear models on structured data more effectively than a capped-depth Random Forest.

11. How to Run

```
pip install pandas numpy scikit-learn matplotlib seaborn scipy statsmodels
jupyter notebook Housing_Case_Study_TreeModels.ipynb
```

12. Repository Contents



File	Description
Housing_Case_Study_TreeModels.ipynb	Main analysis notebook
Housing.xlsx	Source dataset
correlation_heatmap.png	Numeric feature correlation matrix
categorical_boxplots.png	Price distribution by categorical features
residual_histogram.png	Residual distribution (linear model)
actual_vs_predicted.png	Actual vs. predicted price (linear model, test set)
decision_tree.png	Visualized decision tree regressor
rf_feature_importance.png	Random Forest feature importance
model_comparison.png	Train/test R ² comparison across all three models
README.md	This report

13. Conclusion

This case study builds and rigorously compares three regression approaches for predicting Delhi-region housing prices. A carefully refined Multiple Linear Regression model — built through disciplined, statistically-grounded feature selection using p-values and VIF — achieves the best generalization (0.660 test R²), outperforming both a Decision Tree (0.529) and a Random Forest (0.585). Area and bathroom count emerge as the dominant price drivers across every model tested, while several amenities (air conditioning, hot water heating, preferred area, main-road access) contribute smaller but statistically significant premiums. The key takeaway for a data science portfolio: **model sophistication is not a substitute for careful feature engineering and statistical rigor**, particularly on small-to-medium, largely linear datasets like this one.

Tech stack: Python · pandas · numpy · scipy · scikit-learn · matplotlib · seaborn